

词法分析

要实现一个编译器，最开始的一步是对要进行编译的代码进行 `词法分析`。其实词法分析很简单，对于一个程序，要对他的文本进行一个拆分，拆分出来的每一个小单元就称为一个 `token`，每一个token除了包含他原本的内容之外，还有他自己的类别。类别是为了方便后续对token之间进行语法分析。

我的整体设计

目前只考虑词法分析部分，后续这里可能会有一部分问题而需要进行调整。

实现了词法分析之后我的文件结构如下：

```
src
├─ Compiler.java
├─ config.json
├─ lexer
│   ├─ Lexer.java
│   ├─ Token.java
│   ├─ TokenType.java
│   └─ TokenTypeMap.java
└─ utils
    └─ Tools.java
```

这里的词法分析的主要逻辑在 `Lexer.java` 中实现，负责对文本进行拆分，拆分的结果是自己定义的一个 `Token` 类型的对象。在 `utils` 中的的是在这个过程中可能用到的工具类，虽然大多数都没有用到，比如：输入文件路径输出对应的 `String` 类型的文本内容的静态函数等，这样可以方便在 `Lexer` 中调用，可以简洁部分代码逻辑。

Token 类

想了半天感觉还是从 `Token` 这个类说起来比较好。`Token` 是我们词法分析需要输出的结果，每一个 `Token` 类都包含以下基本信息：

1. `Token` 原本自己的字符文本 `lexeme`。
2. `Token` 的类别，这里我把他的类别写成了一个枚举类 `TokenType` 方便进行管理。
3. `Token` 所在的行数，这里要记录 `Token` 所在的行数是方便如果后续判断该 `Token` 有问题时，报错时方便输出其所在行号。

上述信息我将其设置成 `private` 类型，同时只有 `get` 方法供外界获取，对 `Token` 的基本信息进行了一个保护。除此之外，因为实验最后的要求是输出每一个 `Token` 的信息（正确的输出“类别 内容”，错误的输出“错误类型 行号”），为了方便后续输出，我对 `Token` 的 `toString` 方法进行了一个重构，使其会根据其 `TokenType` 的内容进行选择输出不同类型的内容。

TokenType 类

很显然需要有这一个枚举类用来枚举所有 `Token` 的属性。

```
public enum TokenType {  
    IDENFR,      // Ident  
    INTCON,      // int Const  
    CHARCON,     // char const  
    .....  
}
```

就是这么简单，但是如果这么写又有一个问题，比如我现在得到了一个 token，这个 Token 的具体内容是一串英文字母：当然，因为词法中有一些是**标识符**，也有一些是关键字，当我们得到了一个 token 之后该如何通过这个 token 的内容得知对应的 TokenType 类型到底是 IDENFR 还是 WHILETK 这种的呢。很显然只靠这一个 TokenType 类要解决这个问题太乏力了。

TokenTypeMap 类

为了解决上述的问题，我添加了一个 `TokenTypeMap` 类，用来记录这种特殊的**关键字**到 Token 类型的映射关系。

```
public class TokenTypeMap {  
    private static final Map<String, TokenType> keywords = Map.ofEntries(  
        Map.entry("continue", TokenType.CONTINUETK),  
        Map.entry("default", TokenType.DEFAULTTK),  
        .....  
    );  
}
```

通过这种方式，每次我截取出一段字符串之后，便可以用这段字符串在这个 Map 中进行查找，如果找到了就返回对应的 Token 类型，如果没找到就说明这是一个标识符。这样就非常优雅的解决了对于 Token 进行分类的问题。

Lexer.java 实现对于文本的划分

首先，`Lexer` 类中有解析文件对应的文本内容，将所有内容都放到了一个 String 类型中，这就是这个类中的 `source` 属性。在进行对于整个文本进行一遍的处理过程中，还需要一个 `pos` 表示当前处理的位置。同时为了方便记录对应的 Token 的位置，还有一个 `line` 变量记录当前所在行数。

实现对于文本的安全访问

要实现对于文本的划分，其实很重要的一点是要保证 `pos` 始终在我们的掌控之中。为了实现对于文本的安全访问，而不是随便的 `source.charAt(pos)`，我在这个类中设计了几个 private 方法，用来实现对于文件内容的安全访问：

```
// isAtEnd() 函数，用来判断当前位置是否超出了文件的最大长度（越界），如果是返回 true，否则返回 false  
private boolean isAtEnd() {  
    return pos >= source.length();  
}  
  
// peek() 函数，对当前位置的字符进行查看，如果当前位置已经越界了直接返回 '\0'  
private char peek() {  
    if(!isAtEnd()) {  
        return source.charAt(pos);  
    }  
}
```

```

        else return '\0';
    }

    // nextPeek() 函数，对当前位置的下一个字符进行查看，如果当前位置的下一个已经越界了直接返回 '\0'
    private char nextPeek() {
        if(pos+1 >= source.length()) {
            return '\0';
        }
        return source.charAt(pos+1);
    }

    // advance() 函数，返回当前的位置的字符，之后对pos进行++，指向下一个字符。
    private char advance() {
        if(isAtEnd()) return '\0';
        return source.charAt(pos++);
    }

```

这些函数在开始的地方都进行了 `isAtEnd()` 的包装，这样的话即使想要访问的位置已经越界，也不会发生错误，而是返回空 `\0`。经过这样的设计很大程度上方便了后续对于文本的访问，也使得后面的代码可读性明显提高。

文本划分的实现

在完成了上面的对于文本位置的安全访问的实现之后就可以开始真正的进行对文本的划分了。主要的获取下一个Token的方式就是通过下面的方法 `nextToken` 的调用实现的。这个方法先会跳过空白，然后根据读到的第一个字符来进行判断，如果读入的是数字，那只有可能是数字常量，如果读入的是字母或者下划线_那也只有可能是标识符了，剩下的就是一些符号，最后进行对于符号的读。这样就是实现了对于Token的读取了。

```

public Token nextToken() {
    skipWhiteSpace();

    // System.out.println(pos);
    char c = peek();
    if(c == '\0') return null;

    if(Character.isLetter(c) || c == '_') {
        return scanIdentifier();
    }

    if(Character.isDigit(c)) {
        return scanNumber();
    }

    return scanSymbol();
}

```

首先是这个 `skipWhiteSpace` 函数：

```
private void skipWhiteSpace() {
    while(!isAtEnd()) {
        updateLine();
        char ch = peek();
        if(ch == ' ' || ch == '\r' || ch == '\n' || ch == '\t') {
            advance();
        } else {
            break;
        }
    }
}
```

就是一个循环，如果当前是空白字符就继续读下一个，直到非空白字符停止。

先看比较简单的 `scanNumber` 方法：

```
private Token scanNumber() {
    int start = pos;

    while(!isAtEnd() && Character.isDigit(peek())) {
        advance();
    }

    String lexeme = source.substring(start, pos);

    TokenType type = TokenType.INTCON;

    return new Token(type, lexeme, line);
}
```

如果读进来的第一个字符是数字，那么就一直往后读数字，直到第一个不是数字的字符为止，这个时候得到的字符串就是一个 `整数` 类型的 `Token`。在这里使用了 `String` 的 `substring` 的方法。这个地方和 `C++` 有点区别，比如 `s.substring(2,5)` 表示的是 `s` 的第 2 到第 5 个字符(不含第5个)组成的子字符串。而 `C++` 中表示从位置2开始，长度为5的子字符串。

`scanIdentifier` 方法和上面的类似，不同的是对于一个标识符，处理开头是字母或者下划线，其他位置也可以是数字。同时在读完一个标识符之后，再去上面的 `TokenTypeMap` 中查找是否是关键字，根据查找的结果确定 `Token` 的类型。

最后就是 `scanSymbol` 的划分了。因为是一些确定的符号，我这里使用了 `switch` 的形式来进行分支的选择，一方面是效率更高，另一方是看起来也舒服很多。在这个地方我还记录了一个 `boolean` 类型的变量，这个后面会进行说明。

```
private Token scanSymbol() {
    char c = advance();
    boolean isEscape = false; // 记录当前是否会发生转义
    int start ;
    switch(c) {
```

首先是一些确定的字符串，比如说 `+` 这种的，他不会和其他的组合成一个其他的东西，这种就很简单了，直接读完之后返回对应类型的 token 就行了：

```
case '+':
    return new Token(TokenType.PLUS, "+", line);
```

接下来是确定长度，但是可以继续往后拼接的，比如说 `<` 单独就是 LSS，但是它可以后面拼接一个 `=` 变成 `<=`，这个时候就变成了 LEQ 了。这种的处理方式也很简单，由于规则中最长的也就长度为 2 的，只需要读入第一个之后，查看他的下一个是不是能拼接的就行了，如果能拼接就拼接，不能拼接就直接返回当前对应的类型：

```
case '<':
    if(!isAtEnd() && peek() == '=') {
        advance();
        return new Token(TokenType.LEQ, "<=", line);
    }
    return new Token(TokenType.LSS, "<", line);
```

最开始我们使用 `advance` 获得这个位置的字符的，所以现在我们访问这个字符的下一个只需要 `peek` 就行了，而不是 `nextPeek`

然后就是长度不确定的字符串类型了，虽然说读入单引号之后只能有一个字符，但是考虑到我们目前还没有涉及到这个方面的错误处理，这里就先将单引号和双引号当成同一种情况处理了。这里的难点在于如何让这个字符串停下来。对于一般的字符串比如说 `"adsasdi fh"` 我们一般是从左边的双引号开始读，读到右边的第一个双引号结束为止。事实确实如此，可是如果字符串是这样的呢 `"sdhfasdf\""` 这里第二个双引号被前面的 `\` 转义了，这导致按照道理不应在第二个双引号就停下来，而应该在最后一个双引号才停下来。

很朴素的，那我们是不是只要考虑在后面读到双引号的时候前面有没有这个 `\` 把他转义就行了呢？如果转义了就继续往后读，否则就停下来。这样看起来合理，但实际上不可行，比如下面这个例子就会出现问題：`"hasjdfsakj\""`，这里双引号前面的 `\` 被转义了，所以其实这里的双引号没有被转义，到第二个双引号就应该停下来，可是如果按照刚才说的逻辑就不会停下来，而出现问题。

那到底该如何解决呢？就是导入一个前面提到的变量用来标记当前位置是否会进行转义。

```
boolean isEscape = false; // 记录当前是否会发生转义
```

当我们读入到 `\` 的时候就把这个 `isEscape = !isEscape;` 为什么不是直接变为 true？其实原因就是防止出现 `\\` 这种情况。在其他情况，都直接把这个 `isEscape` 置为 false。当我们读到 `"` 的时候，就看当前是否要转义，如果要转义，就继续往后读，否则就停下来，当然不要忘记每次对于这个 `isEscape` 的维护。

```
case '\"':
    start = pos-1;
    while(!isAtEnd()) {
        if(peek() == '\\') {
            isEscape = !isEscape;
            advance();
        }
        if(peek() != '\"') {
            if(isEscape) isEscape=false;
            advance();
        } else if(isEscape) {
```

```

        isEscape=false;
        advance();
    } else {
        advance();
        break;
    }
}
return new Token(TokenType.STRCON, source.substring(start, pos),
line);

```

最后还剩对于注释的处理了。这里的逻辑和前面的类似，主要还是注意 pos 的移动，比如对于 `/**/` 类型的注释，最后要使用 advance 把后面的两个 `*/` 吃掉，配合 `skipWhiteSpace` 让 pos 指向下一个 token 的开始。

```

case '/' :
    if(!isAtEnd()) {
        if(peek() == '/') {
            do {
                advance();
            } while (!isAtEnd() && peek() != '\n');
            return new Token(TokenType.NOTE, "", line);
        } else if(peek() == '*') {
            do {
                if(peek() == '\n') updateLine();
                advance();
                // System.out.println(peek() + " " + nextPeek());
            } while (!isAtEnd() && !(peek() == '*' && nextPeek() ==
'/'));

            advance();
            advance();
            return new Token(TokenType.NOTE, "", line);
        } else {
            return new Token(TokenType.DIV, "/", line);
        }
    }
}

```

对于行数 line 的更新

最后就只剩对于行数 line 的更新了，这个其实很简单，我们首先和 pos 一样写一个调整这个 line 的方法：

```

private void updateLine() {
    if(peek() == '\n') line++;
}

```

注意这里的 line 应该从1开始，没有第0行啊，现在考虑什么时候的 `\n` 会导致行数 +1:

1. 我们跳过空白时读入的 `\n`
2. 在注释中的 `\n`

只有这两种情况。这在前面的代码中已经加入，这样就实现了对于行数 line 的更新。

程序入口 Compiler.java

这里无需多言：

```
public class Compiler {
    public static void main(String[] args) {
        Lexer lexer;
        Token token;
        StringBuilder content = new StringBuilder();
        StringBuilder error = new StringBuilder();

        try (BufferedReader br = new BufferedReader(new
FileReader("testfile.txt"))) {
            String source = Tools.ReadFile(br);
            lexer = new Lexer(source);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }

        try (BufferedWriter fwOut = new BufferedWriter(new
FileWriter("lexer.txt"));
            BufferedWriter fwErr = new BufferedWriter(new
FileWriter("error.txt"))){
            while ((token = lexer.nextToken()) != null) {
                if(token.getType() != TokenType.a)
                    content.append(token.toString());
                else
                    error.append(token.toString());
            }
            if(error.isEmpty())
                fwOut.write(String.valueOf(content));
            fwErr.write(String.valueOf(error));
        } catch (Exception e) {
            System.err.println(e);
        }
    }
}
```

这样我们就完成了词法分析的全部内容。